DAZIA V1

Phone Connectivity Guide

Connect an Android phone to a local AI assistant running on a Lenovo laptop

Target devices: Android 17 phone + Lenovo laptop (Intel Core i3 11th Gen, 12 GB RAM)

V1 interaction: Open the app-like interface, keep the screen on, tap the microphone, speak, and hear DAZIA's response.

Budget target: $0/month in AI API charges by using local software. Internet and electricity costs are separate.

Version 1.0 • 13 October 2026

This guide describes the intended architecture and implementation steps. It is not a claim that the system is already installed or configured.

1. Goal

Use the Android phone as DAZIA's remote interface while the Lenovo laptop runs the local AI model and backend. V1 deliberately avoids always-listening wake-word detection while the phone is locked.

The phone should be able to send text or a voice recording to DAZIA, display the result, and play a spoken response. Local processing is preferred to avoid AI API charges.

2. Architecture

Recommended request flow:

Android app-like web interface → secure private network → FastAPI backend on Lenovo → Ollama local model → validated tool / SQLite → response back to phone

Part	Runs on	Responsibility
Mobile interface	Android browser / installable PWA	Chat screen, microphone button, transcript, response playback.
Private connectivity	Phone + Lenovo	Securely connects devices over home Wi-Fi and, after setup, mobile data.
FastAPI backend	Lenovo	Receives requests, validates inputs, calls local AI and tools.
Ollama model	Lenovo	Understands the request and drafts a response; hardware performance must be tested.
SQLite + scheduler	Lenovo	Stores tasks/reminders and restores them after backend restarts.

3. Two Connectivity Modes

Mode A — Same Wi-Fi (first milestone): phone and Lenovo connect to the same home network. The phone opens DAZIA using the Lenovo's local network address and backend port. This is for initial testing only; bind/firewall settings must be configured safely.

Mode B — Away from home (second milestone): both devices connect to a private networking solution such as Tailscale. The phone uses the Lenovo's private network address. Do not expose FastAPI directly to the public internet or forward its port on the router.

Remote use still depends on the Lenovo being powered on, awake, connected to the internet, and running the backend and model. Laptop sleep or shutdown makes the local AI unavailable.

4. V1 User Experience

1. Open DAZIA on the Android phone. For V1, the screen must be on and the user taps the microphone.
2. The browser/app requests microphone permission and captures the command.
3. The app sends the recording or transcript to the Lenovo through the private connection.
4. The backend transcribes audio locally if audio upload is enabled, then sends text to the local model.
5. The backend validates any requested action, such as creating a reminder.
6. The phone displays the answer and can play the response using local TTS or supported device speech playback.

Implementation choice: begin with text chat over the private network. Add push-to-talk voice after the text request/response flow is stable.

5. Recommended Free-First Stack

Component	Suggested option	Notes
Backend API	Python + FastAPI	Run on localhost during development; later allow private-network access only.
Local LLM	Ollama + small quantized model	Test speed and RAM usage before settling on a model.
Database	SQLite	Stores reminders, preferences, and optional conversation history.
Scheduler	APScheduler + persisted records	Recover pending reminders on restart; account for laptop downtime.
Mobile UI	Responsive web app / PWA	Simpler than native Android for V1; microphone behavior varies by browser.
Remote network	Tailscale or equivalent private networking	Check current personal-use terms and device setup before relying on it.
Speech-to-text	Local faster-whisper / Whisper	CPU transcription can be slow; benchmark short audio first.
Text-to-speech	Local TTS or phone's built-in speech engine	Select based on voice quality and compatibility.

6. Security Requirements

- Do not expose the FastAPI port directly to the public internet or create a router port-forward for it.
- Use a private network connection for remote access and add application-level authentication as an additional layer.
- Bind to localhost during initial development. Only change binding after configuring firewall rules and authentication.
- Do not hard-code passwords, OAuth tokens, or API keys. Keep secrets out of GitHub.
- Use HTTPS where required by browser microphone APIs; secure private networking may help, but verify the browser's secure-context requirements.
- Treat every incoming request as untrusted. Validate request sizes, input types, and tool arguments.
- Require confirmation before sending emails/messages or performing destructive actions.
- Avoid logging voice recordings, access tokens, or sensitive message contents by default.

7. Practical Build Roadmap

Step	Task	Success check
1	Run Ollama and one small model on Lenovo.	A simple prompt completes without exhausting RAM.
2	Build FastAPI /health and /chat endpoints.	Local browser can send text and receive a response.
3	Build responsive mobile chat page.	Phone can open the page on the same Wi-Fi.
4	Configure firewall and private-network access.	Phone can reach the backend securely; no public port exposure.
5	Add SQLite-backed reminders.	Create/list/complete reminder; it survives restart.
6	Add push-to-talk and audio response.	Phone can send a short voice command and play a reply.
7	Test over mobile data.	Works when the phone is away from home and Lenovo remains online.
8	Add Outlook integration if supported.	OAuth works and read-only access is confirmed before any send action.

8. Troubleshooting Checklist

- Phone cannot connect on home Wi-Fi: confirm both devices are on the same network, verify the Lenovo's current local IP, check Windows Firewall, and confirm FastAPI is listening on the intended interface.
- Phone works on Wi-Fi but not mobile data: verify the private-network app is active on both devices, both devices are signed in to the same authorized private network, and the Lenovo is online.
- Connection works but replies are slow: use a smaller quantized model, reduce context length, and close heavy applications.
- Microphone button fails: check browser permission, use a supported browser, and verify the page is in a secure context.
- Reminders do not fire when laptop is asleep: this is expected for a laptop-hosted service; add missed-reminder recovery on startup or later use an always-on host.

9. Known Limitations

- V1 does not listen for “Hey Dazia” while the phone is locked.
- A PWA may have browser-specific microphone and notification limitations. If reliable native behavior is required, a native Android app can be a later phase.
- The Lenovo's CPU/RAM may limit model speed. Local inference is free in API terms but not necessarily instantaneous.
- Push notifications may not be reliable when the backend or laptop is asleep/offline.
- Outlook and especially Teams features depend on Microsoft account type, API availability, and permissions; verify them separately.
- The remote network tool's current free tier and terms should be checked at implementation time.

10. Acceptance Criteria

- A text request from the phone receives a response from the Lenovo's local model over home Wi-Fi.
- The same request works over mobile data through the private network while the Lenovo is awake and online.
- No paid AI API is called by default.
- The phone can submit a short voice command when the screen is on and receive a spoken/text reply.
- Reminders are stored in SQLite and survive backend restarts.
- The backend is not publicly exposed through router port forwarding.
- Errors are displayed clearly when the Lenovo is offline or unreachable.

11. Coding Assistant Prompt

Copy this prompt into your coding assistant along with the DAZIA main build specification:

Build the phone connectivity layer for DAZIA V1. My Android phone is running Android 17 and my Lenovo laptop has an Intel Core i3 11th Gen CPU and 12 GB RAM. Keep the target at $0/month in AI API charges. V1 uses a mobile-friendly web interface/PWA with the screen on and tap-to-talk; do not implement locked-screen wake-word listening. Run the local model with Ollama on the Lenovo and use FastAPI as the backend. Start with text chat on the same Wi-Fi, then configure secure remote access over mobile data using a private networking solution such as Tailscale. Do not expose FastAPI directly to the public internet or use router port forwarding. Add authentication, input validation, sensible timeouts, and clear offline errors. Use SQLite for reminders and ensure they survive backend restarts. Add voice only after text connectivity works. Verify all steps and report any account, browser, or operating-system limitations instead of assuming they are solved. Make small, testable changes and do not overwrite existing files without asking.

12. Official Resources

- FastAPI: https://fastapi.tiangolo.com/

- Ollama: https://ollama.com/
- Tailscale: https://tailscale.com/
- Android microphone permissions: https://developer.android.com/media/grow/media-projection (check current Android documentation for the exact microphone/foreground-service rules; this link is not a complete microphone guide)
- Progressive Web Apps: https://web.dev/learn/pwa/

Re-check official documentation during implementation because operating-system rules, service terms, and browser requirements can change.